

طراحی الگوریتم‌ها – الگوریتم‌های گراف

Introduction to Algorithm

مهدی جوانمردی

فصل ۲۲ کتاب – الگوریتم‌های پایه گراف

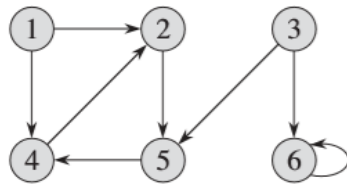
- ۲۲.۱: نمایش گراف
- ۲۲.۲: جستجوی اول سطح
- ۲۲.۳: جستجوی اول عمق
- ۲۲.۴: مرتب‌سازی توپولوژیکی

VI Graph Algorithms

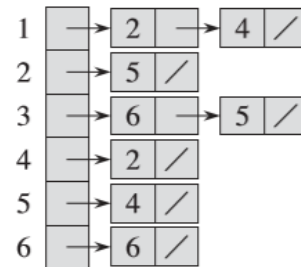
	Introduction	587
22	Elementary Graph Algorithms	589
22.1	Representations of graphs	589
22.2	Breadth-first search	594
22.3	Depth-first search	603
22.4	Topological sort	612

گراف و نحوه نمایش آن

- یک گراف را معمولاً به صورت دو مجموعه رئوس و یال‌ها نمایش می‌دهیم $G(V, E)$
- گراف جهت دار: گرافی است که یال‌ها جهت دارند و یال بصورت یک زوج مرتب بیان میشود



(a)



(b)

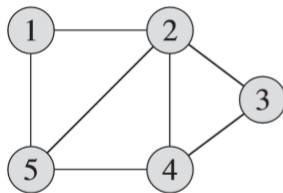
لیست همسایگی

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

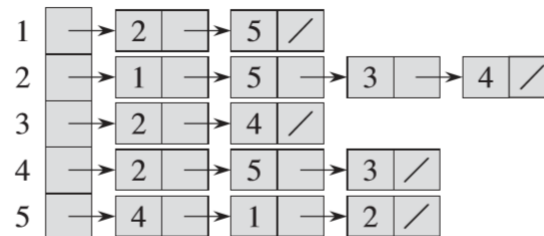
(c)

ماتریس همسایگی

- گراف بدون جهت: در صورت وجود یک رابطه بین دو گره حتماً بر عکس آن نیز وجود دارد



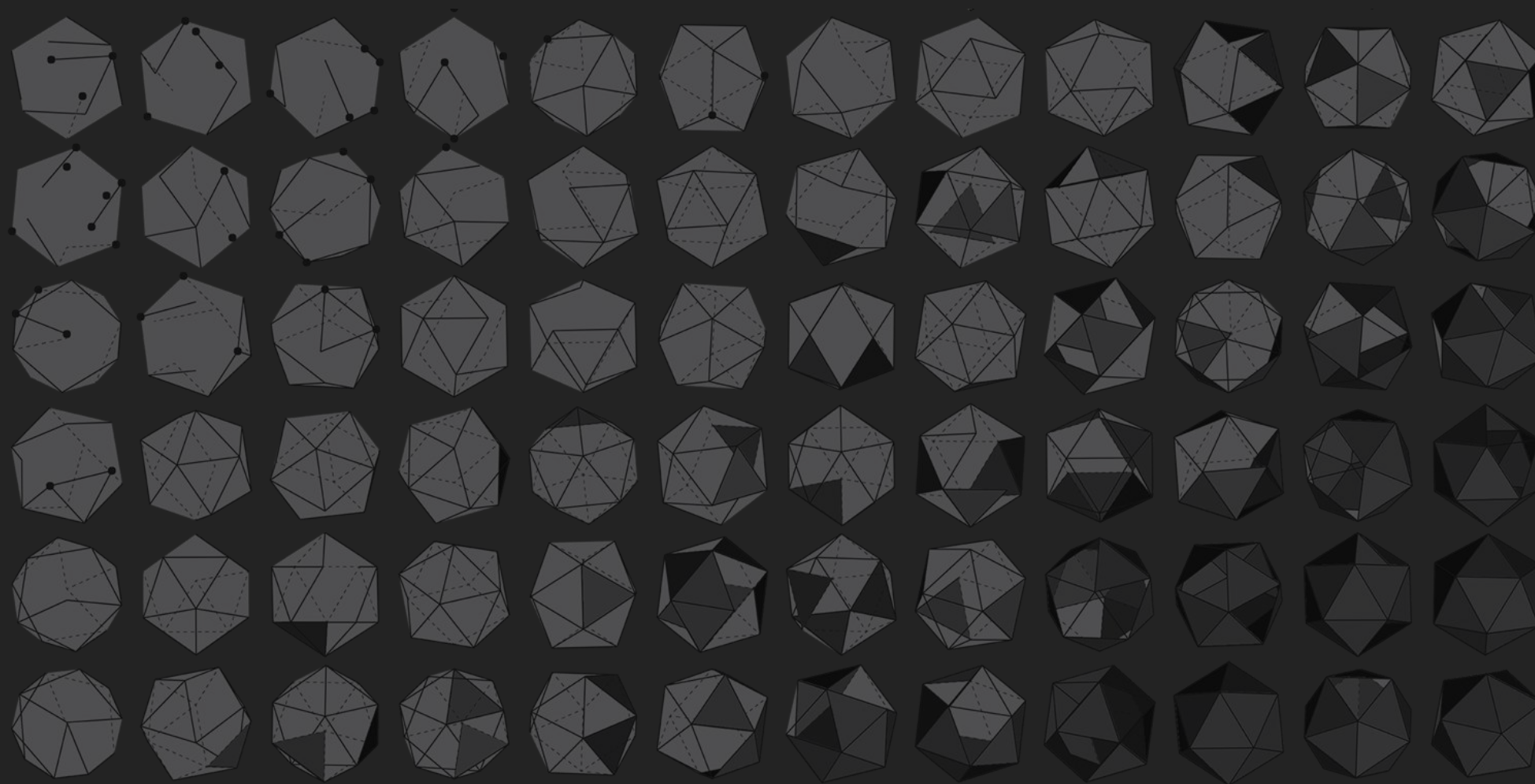
(a)



(b)

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

(c)



جستجوی اول سطح

الگوریتم جستجوی اول سطح (BFS)

- الگوریتم‌های جستجوی گراف بصورت کلی:

- از گره *source* شروع و با پیمایش سیستماتیک یال‌های گراف کشف تمام گره‌های قابل دسترسی از گره منبع

- الگوریتم جستجوی اول سطح یا Breadth-first search

- ورودی: یک گراف $G = (V, E)$ و گره *source*

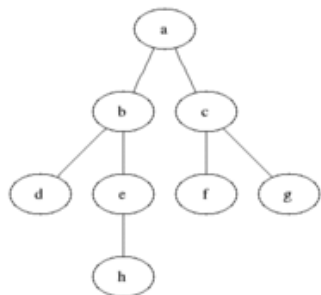
- خروجی: ۱. محاسبه فاصله بین گره‌های قابل دسترسی از گره s تا گره v (کمترین تعداد یال‌ها)

- ۲. درخت اول سطح (breadth-first tree) با ریشه s که شامل تمام گره‌های قابل دسترسی از s هستند

مسیر ساده از s به گره دلخواه v در درخت اول سطح در واقع کوتاه‌ترین مسیر از گره s به v در گراف G است

- علت نامگذاری:

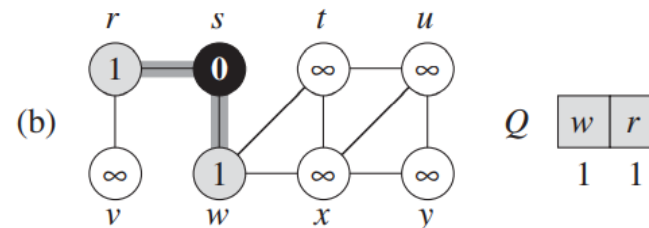
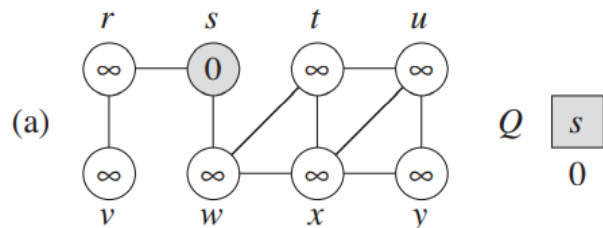
- در BFS همه گره‌های با فاصله K از گره *source* حتما قبل از گره‌هایی با فاصله $K + 1$ مشاهده می‌شوند



http://en.wikipedia.org/wiki/Image:Animated_BFS.gif

الگوریتم جستجوی اول سطح - نحوه عملکرد

- در روند اجرای BFS گره ها به سه رنگ سفید - طوسی - مشکی رنگ می‌شوند
- ابتدای کار رنگ تمام گره‌ها **سفید** می‌باشند و در ادامه شاید **طوسی** و سپس **مشکی** شوند
- هر گره وقتی برای اولین بار مشاهده می‌شود به رنگ **غیر سفید** در می‌آید
- به همین دلیل گره‌های **طوسی** و **مشکی** گره‌هایی هستند که مشاهده شده‌اند
- دلیل رنگ‌گذاری متفاوت طوسی و مشکی دستیابی به هدف **سطح اول** در روند جستجو می‌باشد
- استفاده از ساختمان داده صف (queue) در روند جستجو



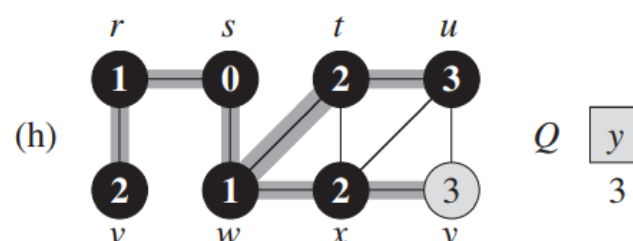
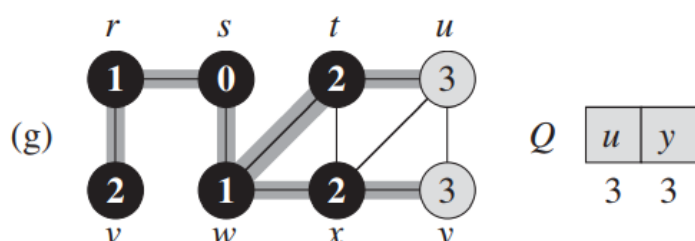
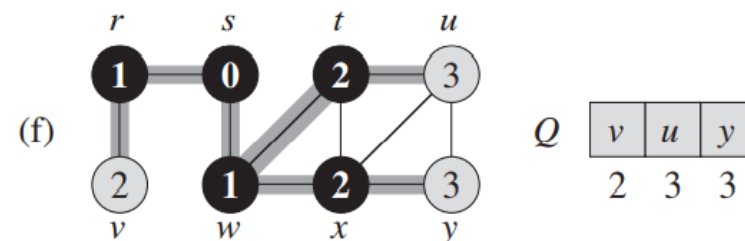
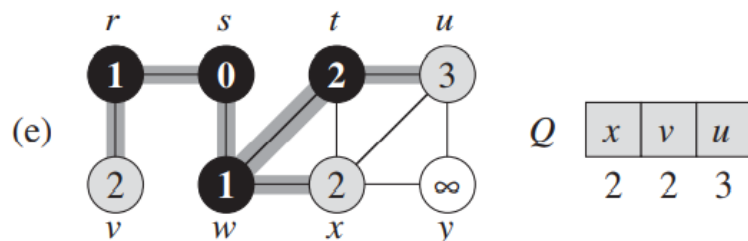
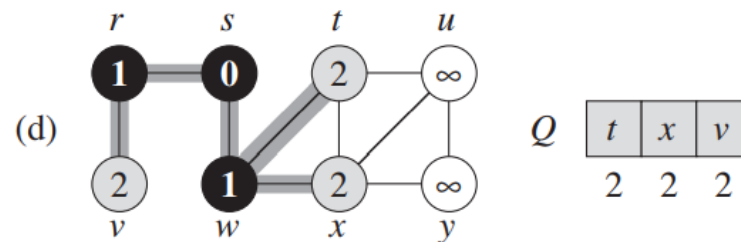
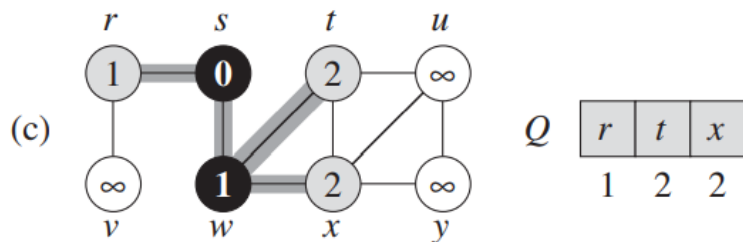
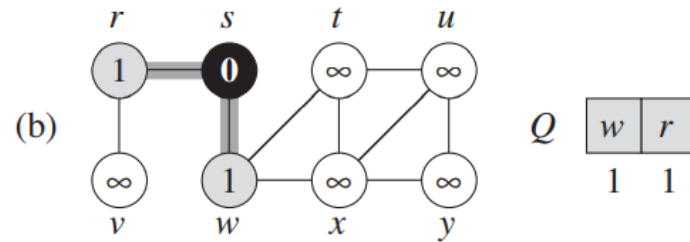
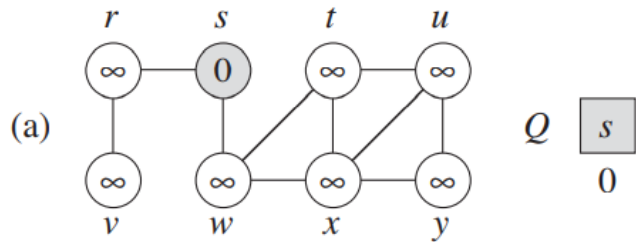
الگوریتم جستجوی اول سطح - شبه کد

BFS(G, s)

```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
```

```
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

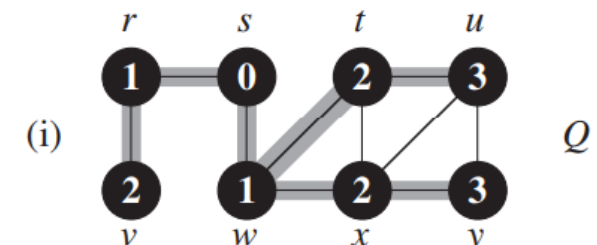
الگوریتم جستجوی اول سطح - مثال



BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = WHITE$ 
3       $u.d = \infty$ 
4       $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = DEQUEUE(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == WHITE$ 
14              $v.color = GRAY$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = BLACK$ 
    
```



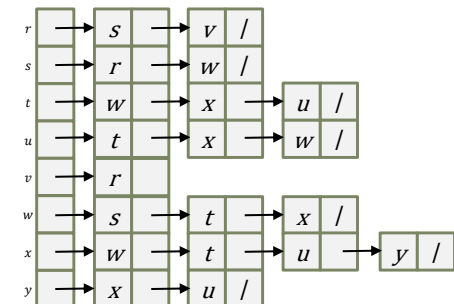
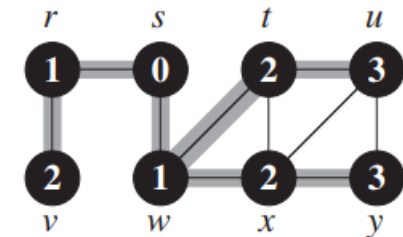
تحلیل زمانی BFS

$$G = (V, E)$$

استفاده از روش تجمیعی برای محاسبه
مرتب‌بندی زمانی الگوریتم

BFS(G, s)

1	for each vertex $u \in G.V - \{s\}$	10	while $Q \neq \emptyset$
2	$u.color = \text{WHITE}$	11	$u = \text{DEQUEUE}(Q)$
3	$u.d = \infty$	12	for each $v \in G.Adj[u]$
4	$u.\pi = \text{NIL}$	13	if $v.color == \text{WHITE}$
5	$s.color = \text{GRAY}$	14	$v.color = \text{GRAY}$
6	$s.d = 0$	15	$v.d = u.d + 1$
7	$s.\pi = \text{NIL}$	16	$v.\pi = u$
8	$Q = \emptyset$	17	$\text{ENQUEUE}(Q, v)$
9	$\text{ENQUEUE}(Q, s)$	18	$u.color = \text{BLACK}$



- Initialization overhead $O(V)$
- Enqueue and Dequeue happen only once for each node. – $O(V)$.
- Sum of the lengths of adjacency lists – $\theta(E)$ (for a directed graph)

Total runtime $O(V + E)$

BFS و کوتاه‌ترین مسیر

• فاصله کوتاه‌ترین مسیر از s به v را با $\delta(s, v)$ نشان می‌دهیم

• اثبات محاسبه فاصله کوتاه‌ترین مسیر در BFS در $v.d$: $v.d = \delta(s, v)$

Lemma 22.1

Let $G = (V, E)$ be a directed or undirected graph, and let $s \in V$ be an arbitrary vertex. Then, for any edge $(u, v) \in E$,

$$\delta(s, v) \leq \delta(s, u) + 1.$$

Lemma 22.2

Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then upon termination, for each vertex $v \in V$, the value $v.d$ computed by BFS satisfies $v.d \geq \delta(s, v)$.

Lemma 22.3

Suppose that during the execution of BFS on a graph $G = (V, E)$, the queue Q contains the vertices $\langle v_1, v_2, \dots, v_r \rangle$, where v_1 is the head of Q and v_r is the tail. Then, $v_r.d \leq v_1.d + 1$ and $v_i.d \leq v_{i+1}.d$ for $i = 1, 2, \dots, r - 1$.

Corollary 22.4

Suppose that vertices v_i and v_j are enqueued during the execution of BFS, and that v_i is enqueued before v_j . Then $v_i.d \leq v_j.d$ at the time that v_j is enqueued.

Theorem 22.5 (Correctness of breadth-first search)

Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then, during its execution, BFS discovers every vertex $v \in V$ that is reachable from the source s , and upon termination, $v.d = \delta(s, v)$ for all $v \in V$. Moreover, for any vertex $v \neq s$ that is reachable from s , one of the shortest paths from s to v is a shortest path from s to $v.\pi$ followed by the edge $(v.\pi, v)$.

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

BFS و کوتاه‌ترین مسیر

- فاصله کوتاه‌ترین مسیر از s به v را با $\delta(s, v)$ نشان می‌دهیم
- اثبات محاسبه فاصله کوتاه‌ترین مسیر در BFS در $v.d$: $v.d = \delta(s, v)$

Lemma 22.1

Let $G = (V, E)$ be a directed or undirected graph, and let $s \in V$ be an arbitrary vertex. Then, for any edge $(u, v) \in E$,

$$\delta(s, v) \leq \delta(s, u) + 1.$$

Proof If u is reachable from s , then so is v . In this case, the shortest path from s to v cannot be longer than the shortest path from s to u followed by the edge (u, v) , and thus the inequality holds. If u is not reachable from s , then $\delta(s, u) = \infty$, and the inequality holds. ■

We want to show that BFS properly computes $v.d = \delta(s, v)$ for each vertex $v \in V$. We first show that $v.d$ bounds $\delta(s, v)$ from above.

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

BFS و کوتاه‌ترین مسیر

- فاصله کوتاه‌ترین مسیر از s به v را با $\delta(s, v)$ نشان می‌دهیم
- اثبات محاسبه فاصله کوتاه‌ترین مسیر در BFS در $v.d$: $v.d = \delta(s, v)$

Lemma 22.2

Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then upon termination, for each vertex $v \in V$, the value $v.d$ computed by BFS satisfies $v.d \geq \delta(s, v)$.

Proof We use induction on the number of ENQUEUE operations. Our inductive hypothesis is that $v.d \geq \delta(s, v)$ for all $v \in V$.

The basis of the induction is the situation immediately after enqueueing s in line 9 of BFS. The inductive hypothesis holds here, because $s.d = 0 = \delta(s, s)$ and $v.d = \infty \geq \delta(s, v)$ for all $v \in V - \{s\}$.

For the inductive step, consider a white vertex v that is discovered during the search from a vertex u . The inductive hypothesis implies that $u.d \geq \delta(s, u)$. From the assignment performed by line 15 and from Lemma 22.1, we obtain

$$\begin{aligned} v.d &= u.d + 1 \\ &\geq \delta(s, u) + 1 \\ &\geq \delta(s, v). \end{aligned}$$

Vertex v is then enqueued, and it is never enqueued again because it is also grayed and the **then** clause of lines 14–17 is executed only for white vertices. Thus, the value of $v.d$ never changes again, and the inductive hypothesis is maintained. ■

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 

```


BFS و کوتاه‌ترین مسیر

• فاصله کوتاه‌ترین مسیر از s به v را با $\delta(s, v)$ نشان می‌دهیم

• اثبات محاسبه فاصله کوتاه‌ترین مسیر در BFS در $v.d$: $v.d = \delta(s, v)$

Lemma 22.3

Suppose that during the execution of BFS on a graph $G = (V, E)$, the queue Q contains the vertices $\langle v_1, v_2, \dots, v_r \rangle$, where v_1 is the head of Q and v_r is the tail. Then, $v_r.d \leq v_1.d + 1$ and $v_i.d \leq v_{i+1}.d$ for $i = 1, 2, \dots, r - 1$.

Proof The proof is by induction on the number of queue operations. Initially, when the queue contains only s , the lemma certainly holds.

For the inductive step, we must prove that the lemma holds after both dequeuing and enqueueing a vertex. If the head v_1 of the queue is dequeued, v_2 becomes the new head. (If the queue becomes empty, then the lemma holds vacuously.) By the inductive hypothesis, $v_1.d \leq v_2.d$. But then we have $v_r.d \leq v_1.d + 1 \leq v_2.d + 1$, and the remaining inequalities are unaffected. Thus, the lemma follows with v_2 as the head.

In order to understand what happens upon enqueueing a vertex, we need to examine the code more closely. When we enqueue a vertex v in line 17 of BFS, it becomes v_{r+1} . At that time, we have already removed vertex u , whose adjacency list is currently being scanned, from the queue Q , and by the inductive hypothesis, the new head v_1 has $v_1.d \geq u.d$. Thus, $v_{r+1}.d = v.d = u.d + 1 \leq v_1.d + 1$. From the inductive hypothesis, we also have $v_r.d \leq u.d + 1$, and so $v_r.d \leq u.d + 1 = v.d = v_{r+1}.d$, and the remaining inequalities are unaffected. Thus, the lemma follows when v is enqueue. ■

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```


BFS و کوتاه‌ترین مسیر

- فاصله کوتاه‌ترین مسیر از s به v را با $\delta(s, v)$ نشان می‌دهیم
- اثبات محاسبه فاصله کوتاه‌ترین مسیر در BFS در $v.d$: $v.d = \delta(s, v)$

Corollary 22.4

Suppose that vertices v_i and v_j are enqueued during the execution of BFS, and that v_i is enqueued before v_j . Then $v_i.d \leq v_j.d$ at the time that v_j is enqueued.

Proof Immediate from Lemma 22.3 and the property that each vertex receives a finite d value at most once during the course of BFS. ■

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 

```

BFS و کوتاه‌ترین مسیر

$$v.d = \delta(s, v)$$

Proof Assume, for the purpose of contradiction, that some vertex receives a d value not equal to its shortest-path distance. Let v be the vertex with minimum $\delta(s, v)$ that receives such an incorrect d value; clearly $v \neq s$. By Lemma 22.2, $v.d \geq \delta(s, v)$, and thus we have that $v.d > \delta(s, v)$. Vertex v must be reachable from s , for if it is not, then $\delta(s, v) = \infty \geq v.d$. Let u be the vertex immediately preceding v on a shortest path from s to v , so that $\delta(s, v) = \delta(s, u) + 1$. Because $\delta(s, u) < \delta(s, v)$, and because of how we chose v , we have $u.d = \delta(s, u)$. Putting these properties together, we have

$$v.d > \delta(s, v) = \delta(s, u) + 1 = u.d + 1. \quad (22.1)$$

Now consider the time when BFS chooses to dequeue vertex u from Q in line 11. At this time, vertex v is either white, gray, or black. We shall show that in each of these cases, we derive a contradiction to inequality (22.1). If v is white, then line 15 sets $v.d = u.d + 1$, contradicting inequality (22.1). If v is black, then it was already removed from the queue and, by Corollary 22.4, we have $v.d \leq u.d$, again contradicting inequality (22.1). If v is gray, then it was painted gray upon dequeuing some vertex w , which was removed from Q earlier than u and for which $v.d = w.d + 1$. By Corollary 22.4, however, $w.d \leq u.d$, and so we have $v.d = w.d + 1 \leq u.d + 1$, once again contradicting inequality (22.1).

Thus we conclude that $v.d = \delta(s, v)$ for all $v \in V$. All vertices v reachable from s must be discovered, for otherwise they would have $\infty = v.d > \delta(s, v)$. To conclude the proof of the theorem, observe that if $v.\pi = u$, then $v.d = u.d + 1$. Thus, we can obtain a shortest path from s to v by taking a shortest path from s to $v.\pi$ and then traversing the edge $(v.\pi, v)$.

Theorem 22.5 (Correctness of breadth-first search)

Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then, during its execution, BFS discovers every vertex $v \in V$ that is reachable from the source s , and upon termination, $v.d = \delta(s, v)$ for all $v \in V$. Moreover, for any vertex $v \neq s$ that is reachable from s , one of the shortest paths from s to v is a shortest path from s to $v.\pi$ followed by the edge $(v.\pi, v)$.

BFS(G, s)

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

درخت اول سطح Breadth-first tree

$G = (V, E)$ with source s .

$G_\pi = (V_\pi, E_\pi)$

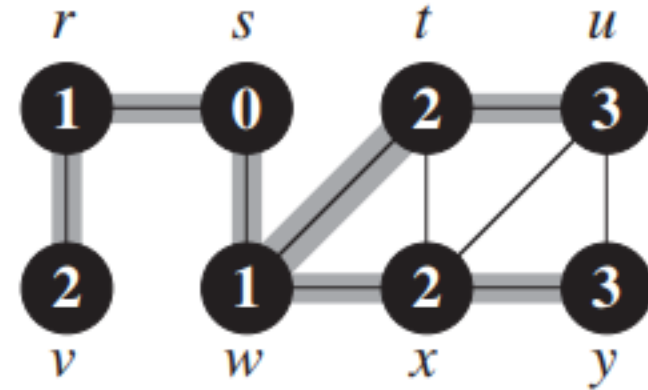
$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$

$E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}$

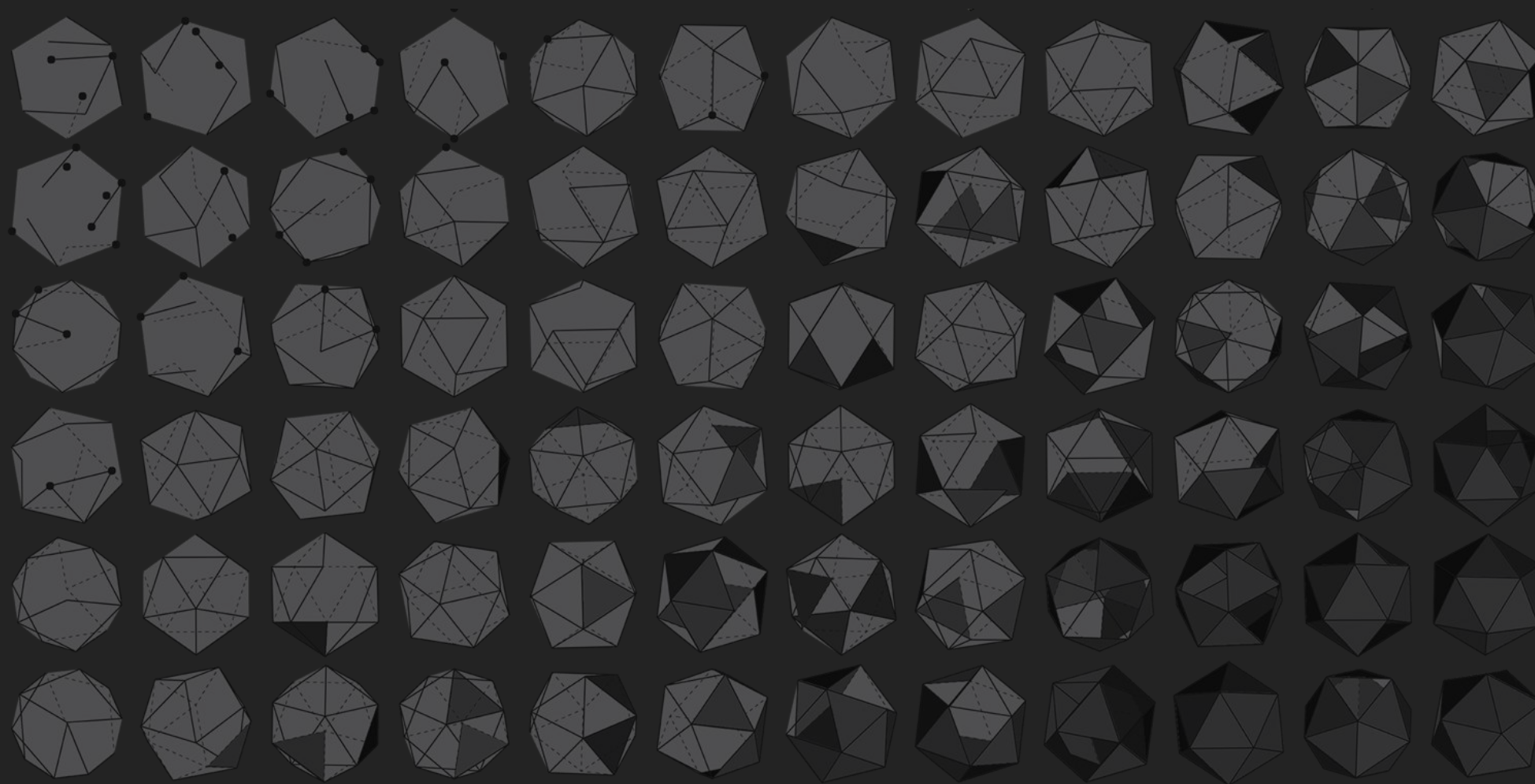
PRINT-PATH(G, s, v)

```

1  if  $v == s$ 
2      print  $s$ 
3  elseif  $v.\pi == \text{NIL}$ 
4      print "no path from"  $s$  "to"  $v$  "exists"
5  else PRINT-PATH( $G, s, v.\pi$ )
6      print  $v$ 
    
```



$$|E_\pi| = |V_\pi| - 1$$



جستجوی اول عمق

الگوریتم جستجوی اول عمق (DFS)

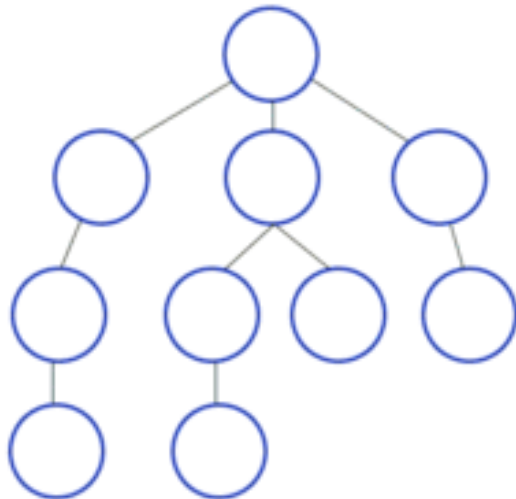
• الگوریتم جستجوی اول عمق یا Depth-first search

• مفهوم: جستجوی عمیق‌تر در گراف تا زمانی که امکان دارد (دقیقا عکس جستجوی اول سطح)

• ورودی: یک گراف $G = (V, E)$ (نیازی به گره *source* نداریم)

• خروجی: ۱. زمان‌های کشف $(v.d)$ و پایان $(v.f)$ برای تمام گره‌ها

۲. جنگل اول عمق (depth-first forest)



These timestamps are integers between 1 and $2|V|$

$$u.d < u.f$$

الگوریتم جستجوی اول عمق - شبهه کد

The variable *time* is a global variable

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
    
```

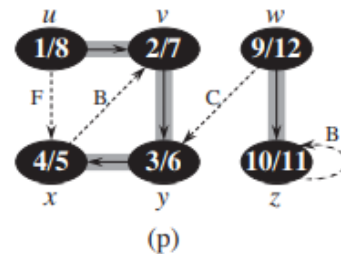
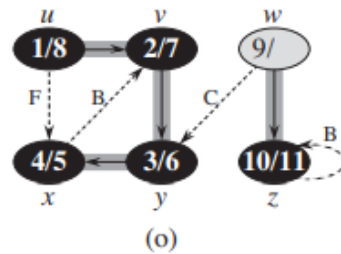
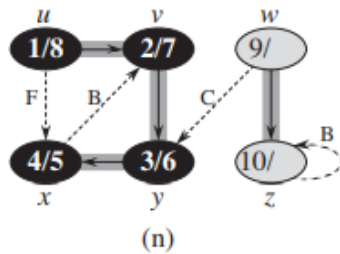
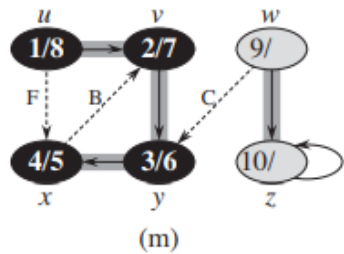
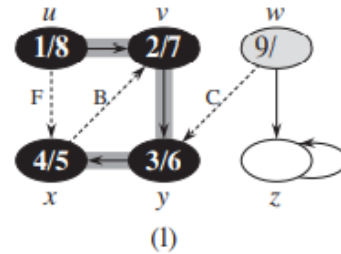
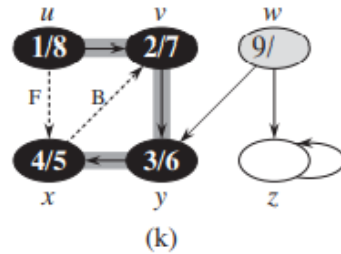
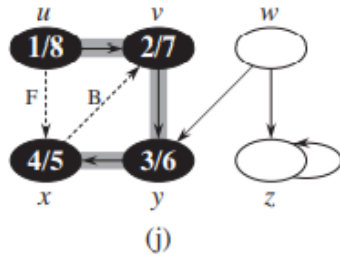
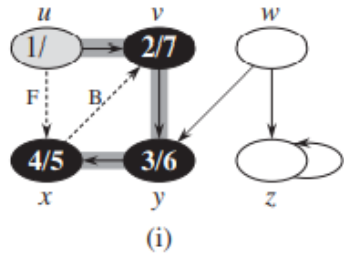
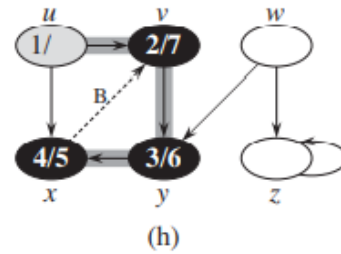
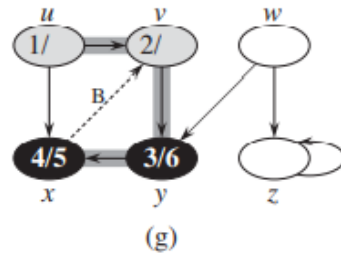
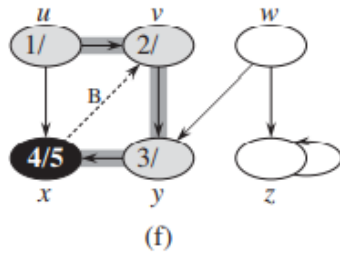
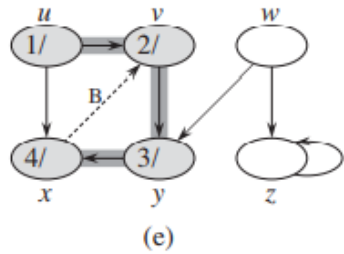
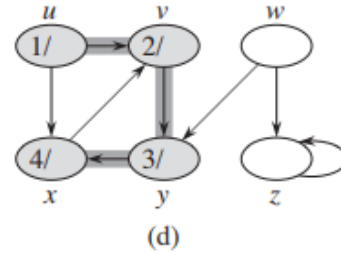
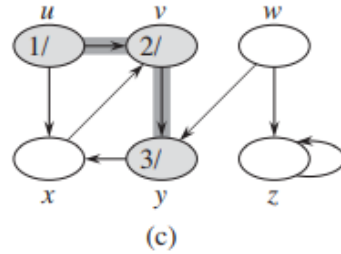
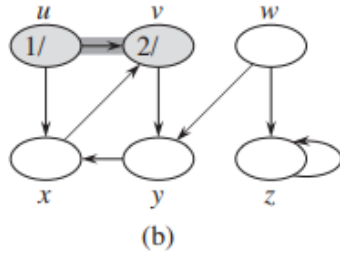
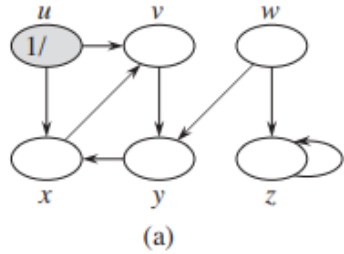
DFS-VISIT(G, u)

```

1   $time = time + 1$                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$           // explore edge  $(u, v)$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$                 // blacken  $u$ ; it is finished
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

Depth first results may depend upon the order in which DFS examines the vertices and visit neighbors

الگوریتم جستجوی اول عمق - مثال



DFS(G)

```

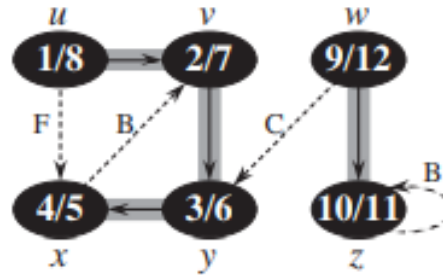
1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
    
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

جنگل اول عمق Depth-first Forest



$G_\pi = (V, E_\pi)$, where

$$E_\pi = \{(v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL}\}$$

$$G = (V, E)$$

تحلیل زمانی DFS

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
    
```

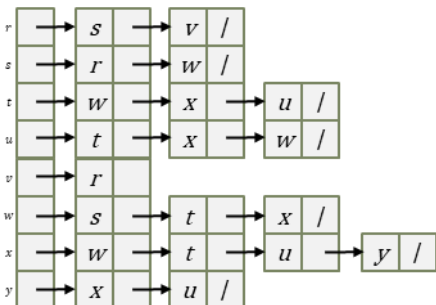
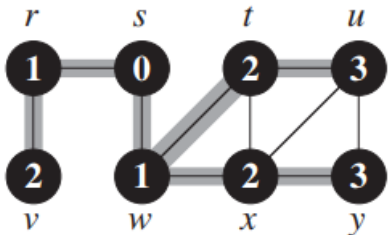
$\Theta(V)$

$\Theta(V)$ (Exclusive of DFS-Visit)

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```



استفاده از روش تجمیعی برای محاسبه
مرتبۀ زمانی الگوریتم

$$\sum_{v \in V} |Adj[v]| = \Theta(E)$$

مجموع لیست همسایگی‌ها:

• مقدار دهی اولیه: $O(V)$

• تعداد فراخوان‌های DFS-VISIT: $O(V)$

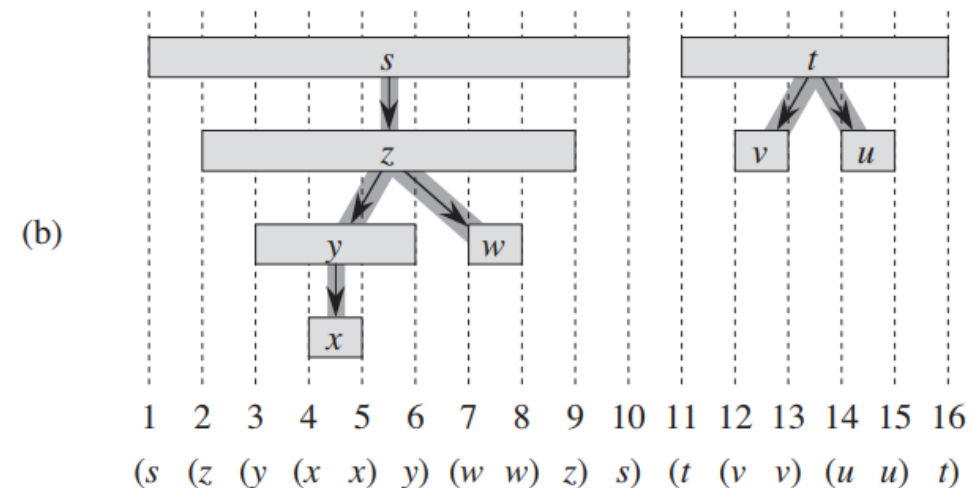
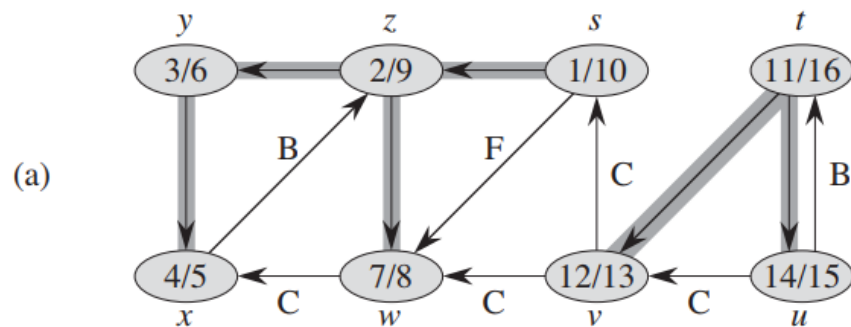
• زمان اجرای DFS: $\Theta(V + E)$

ویژگی‌های DFS

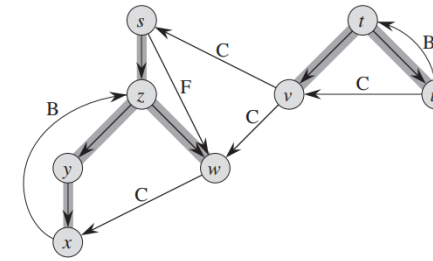
Theorem 22.7 (Parenthesis theorem)

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.



انواع یال‌ها در DFS

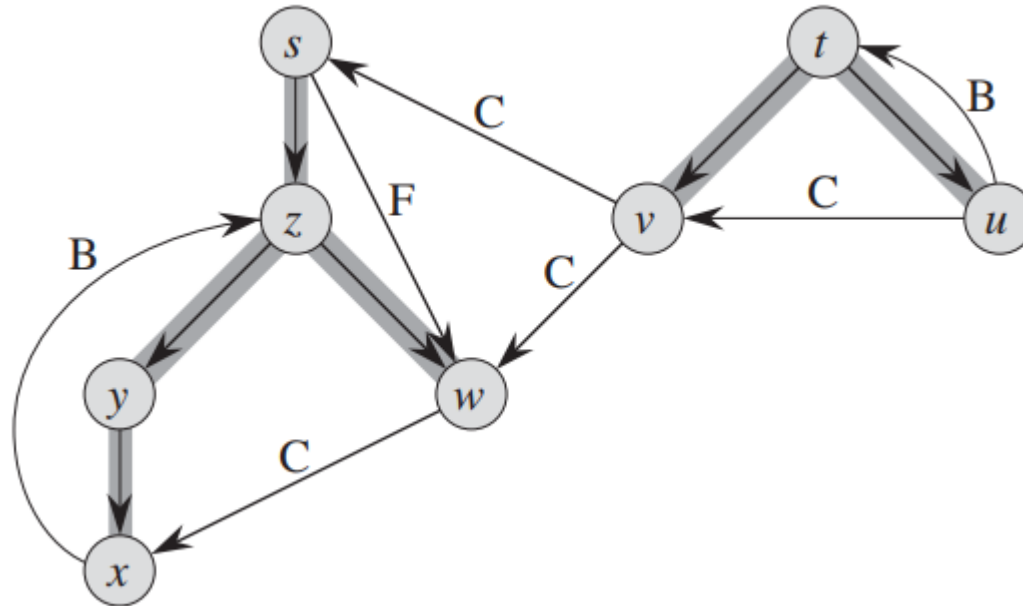


Classification of edges

We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

1. **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
2. **Back edges** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.
3. **Forward edges** are those nontree edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.
4. **Cross edges** are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

انواع یال‌ها در DFS



T: Tree edge

B: Back edge

F: Forward edge

C: Cross edge

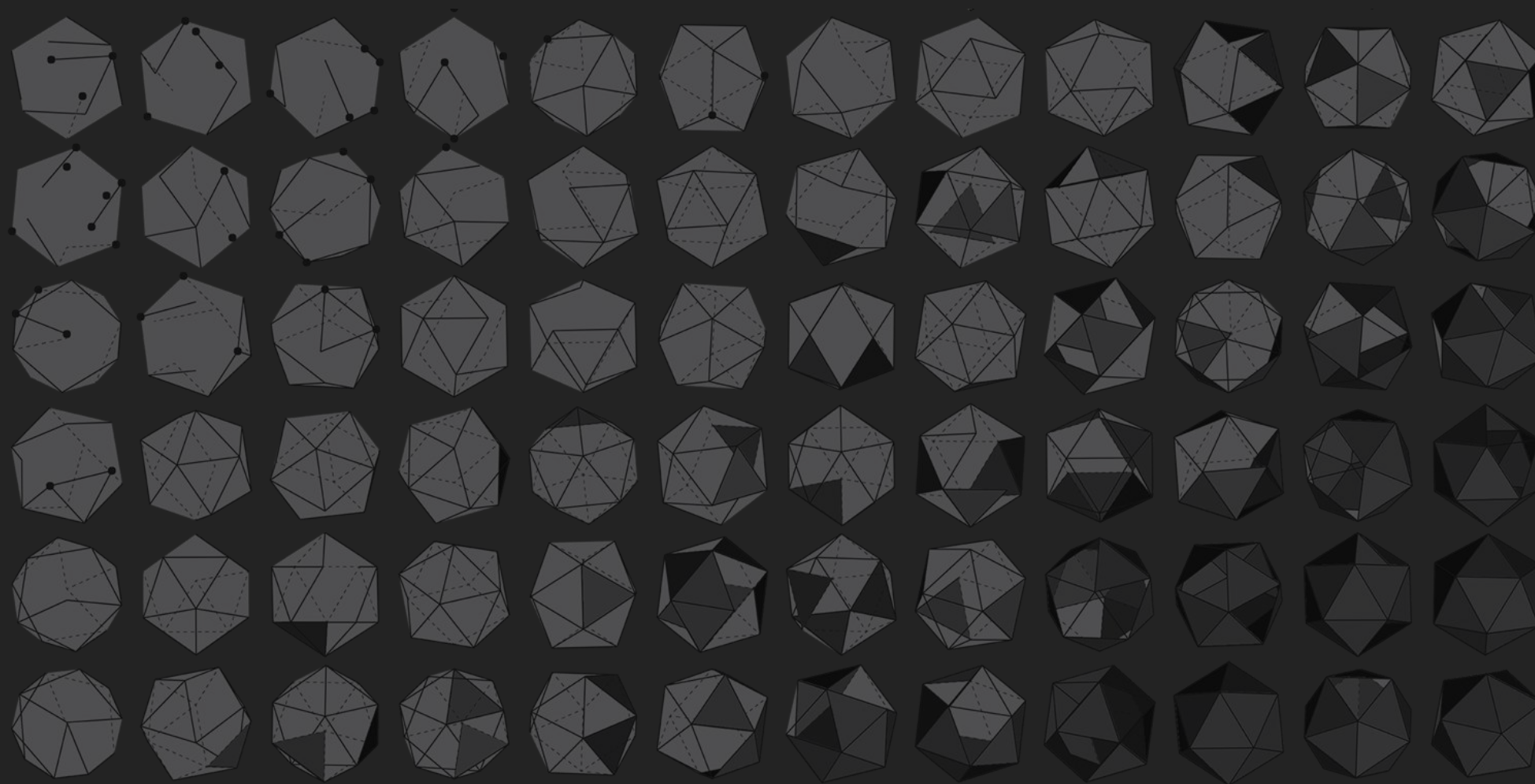
The DFS algorithm has enough information to classify some edges as it encounters them. The key idea is that when we first explore an edge (u, v) , the color of vertex v tells us something about the edge:

1. WHITE indicates a tree edge,
2. GRAY indicates a back edge, and
3. BLACK indicates a forward or cross edge.

edge (u, v) is a forward edge if $u.d < v.d$ and a cross edge if $u.d > v.d$.

BFS and DFS – possible applications

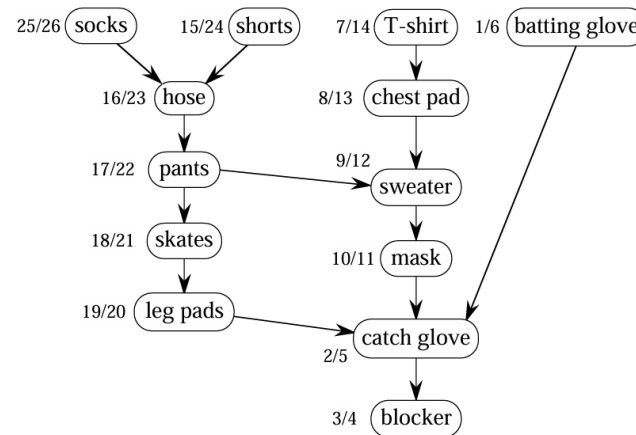
- Exploration algorithms in Artificial Intelligence
- Possible to use in routing / exploration wherever travel is involved. E.g.,
 - I want to explore all the nearest pizza places and want to go to the nearest one with only two intersections.
 - Find distance from my factory to every delivery center.
 - Most of the mapping software (GOOGLE maps, YAHOO(?) maps) should be using these algorithms.
 - Companies like Waste Management, UPS and FedEx?
- Applications of DFS
 - Validation of DAG (directed acyclic graph)
 - Topologically sorting a DAG.
 - List the graph elements in such an order that all the nodes are listed before nodes to which they have outgoing edges.
 - Finding the strongly connected components of a directed graph.
 - A directed graph is called strongly connected if there is a path in each direction between each pair of vertices of the graph.
 - Backtracking for Constraint Satisfaction Problems (CSPs)



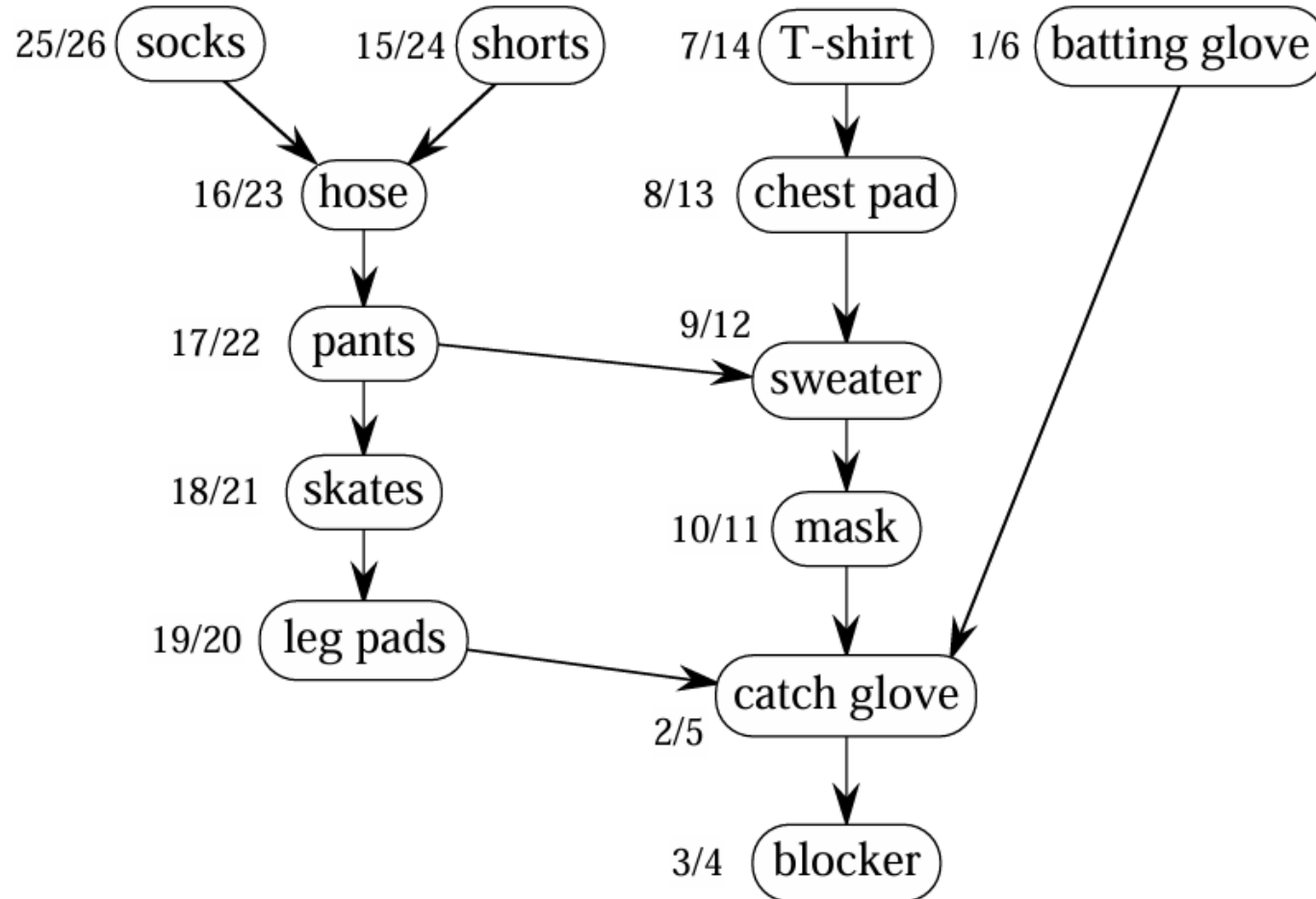
مرتب‌سازی توپولوژیکی

گراف جهت‌دار بدون دور (DAG)

- A directed graph with no cycles.
- Good for modeling processes and structures that have a partial order:
 - $a > b$ and $b > c \rightarrow a > c$
 - But may have a and b such that *neither* $a > b$ nor $b > a$.
- Can always make a total order (*either* $a > b$ or $b > a$ for all $a \neq b$) from a partial order. In fact, that's what a topological sort will do.



گراف جهت‌دار بدون دور (DAG)



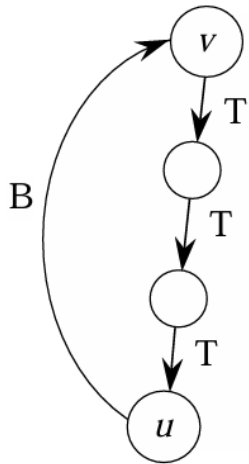
اثبات دور نداشتن گراف جهت‌دار

Lemma

A directed graph G is acyclic if and only if a DFS of G yields no back edges.

Proof $\Rightarrow$: Show that back edge $\Rightarrow$ cycle.

Suppose there is a back edge (u, v) . Then v is ancestor of u in depth-first forest.



(white path theorem). In a depth-first forest of a graph, vertex v is a descendant of vertex u if and only if at the time $d[u]$ when the search discovers u , v can be reached from u along a path consisting entirely of white vertices.

Therefore, there is a path $v \rightsquigarrow u$, so $v \rightsquigarrow u \rightarrow v$ is a cycle.

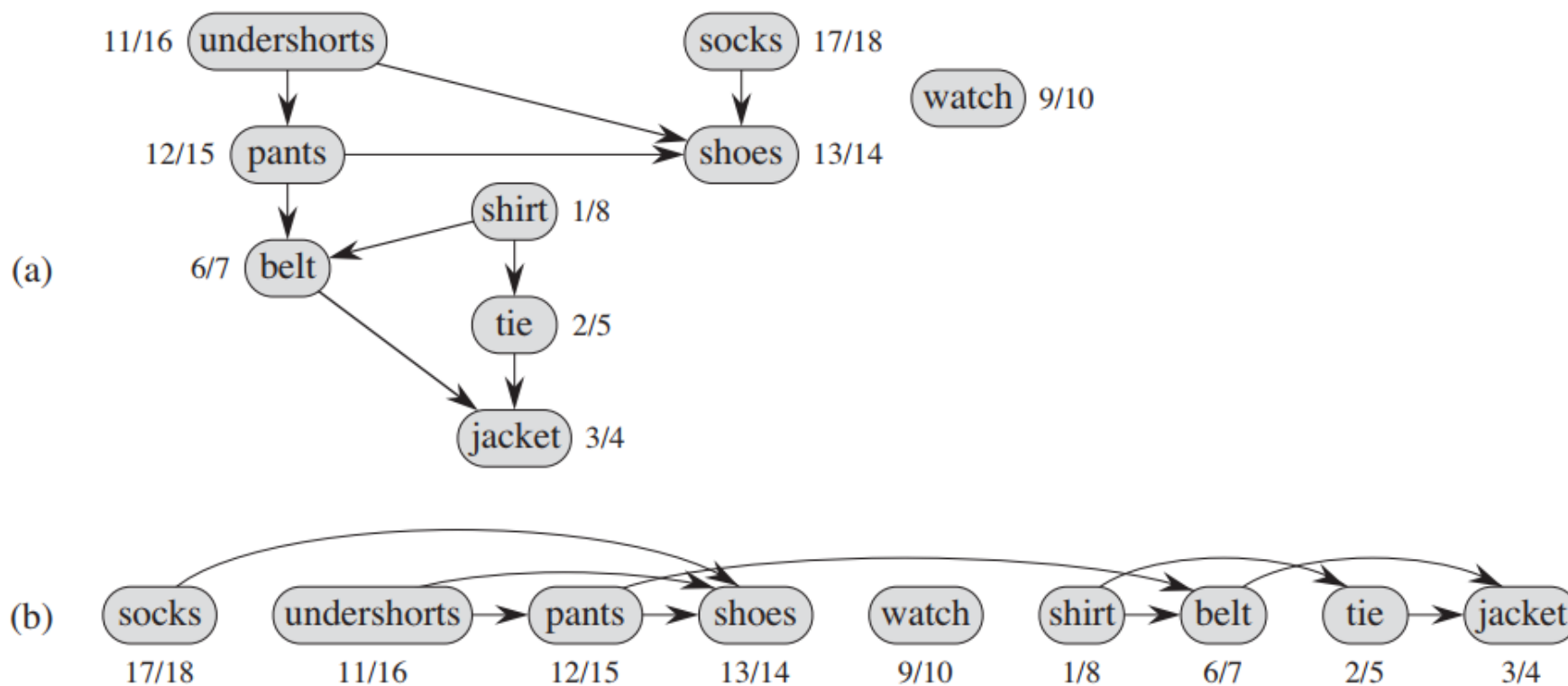
$\Leftarrow$: Show that cycle $\Rightarrow$ back edge.

Suppose G contains cycle c . Let v be the first vertex discovered in c , and let (u, v) be the preceding edge in c . At time $v.d$, vertices of c form a white path $v \rightsquigarrow u$ (since v is the first vertex discovered in c). By white-path theorem, u is descendant of v in depth-first forest. Therefore, (u, v) is a back edge. ■ (lemma)

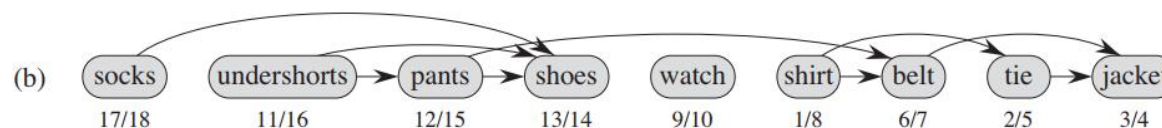
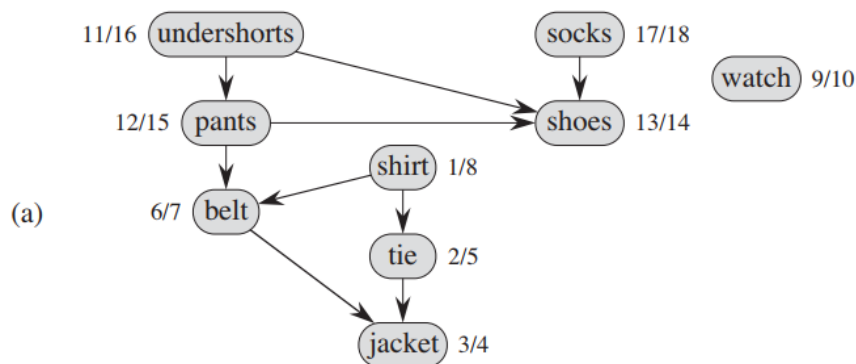
مرتب‌سازی توپولوژیکی یا Topological Sort

یک ترتیب خطی از همه رئوس گراف به‌طوری‌که هر گره قبل از همه گره‌هایی می‌آید که از آن به آن‌ها یال خارج شده‌است

directed acyclic graph, or a “dag”



مرتب‌سازی توپولوژیکی یا Topological Sort



TOPOLOGICAL-SORT(G)

- 1 call DFS(G) to compute finishing times $v.f$ for each vertex v
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices

مرتب‌سازی توپولوژیکی یا Topological Sort

Correctness

Just need to show if $(u, v) \in E$, then $v.f < u.f$.

When we explore (u, v) , what are the colors of u and v ?

- u is gray.
- Is v gray, too?
 - No, because then v would be ancestor of u .
 $\Rightarrow (u, v)$ is a back edge.
 $\Rightarrow$ contradiction of previous lemma (dag has no back edges).
- Is v white?
 - Then becomes descendant of u .
 By parenthesis theorem, $u.d < v.d < \underline{v.f} < u.f$.
- Is v black?
 - Then v is already finished.
 Since we're exploring (u, v) , we have not yet finished u .
 Therefore, $v.f < u.f$.

DFS(G)

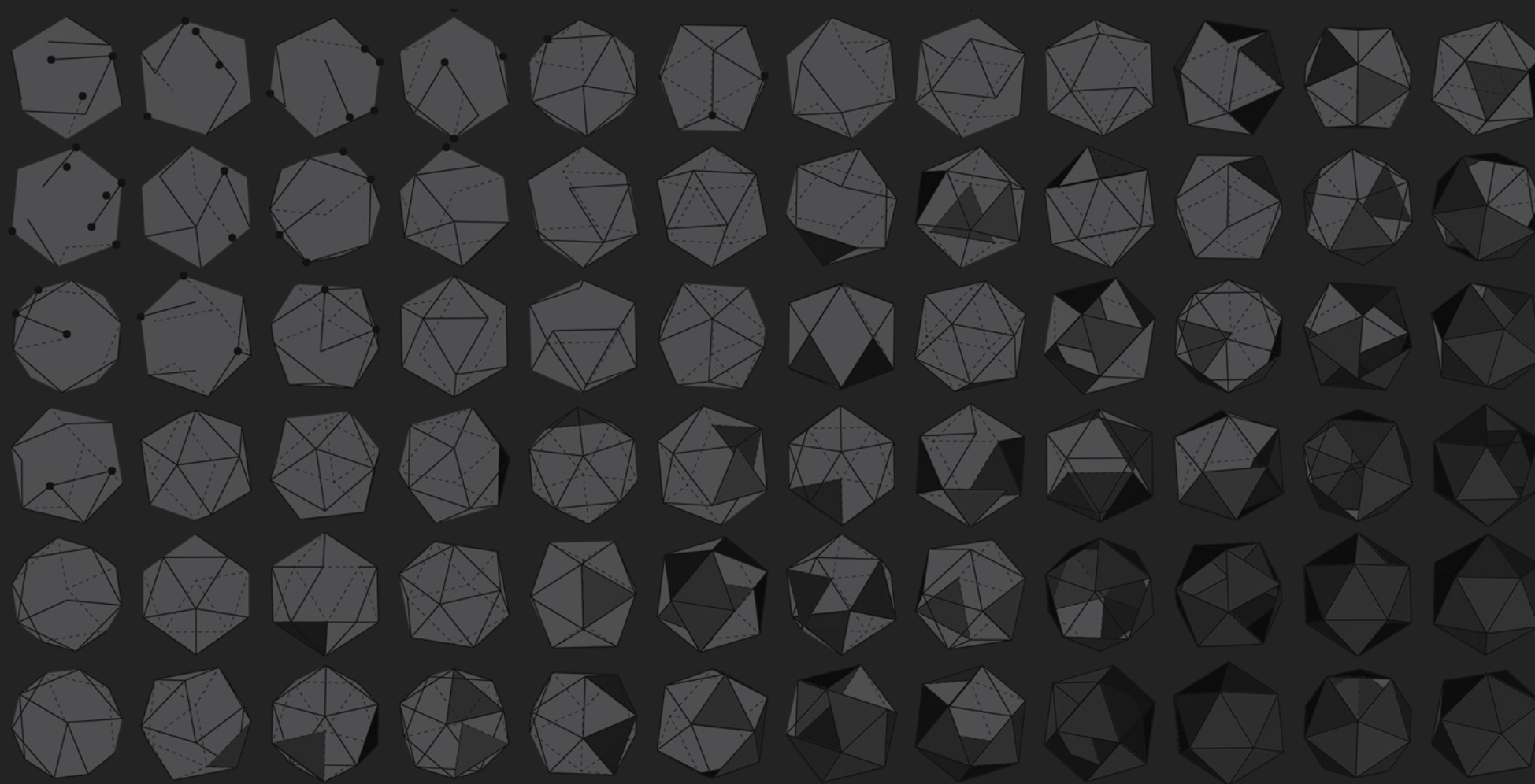
```

1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
  
```

DFS-VISIT(G, u)

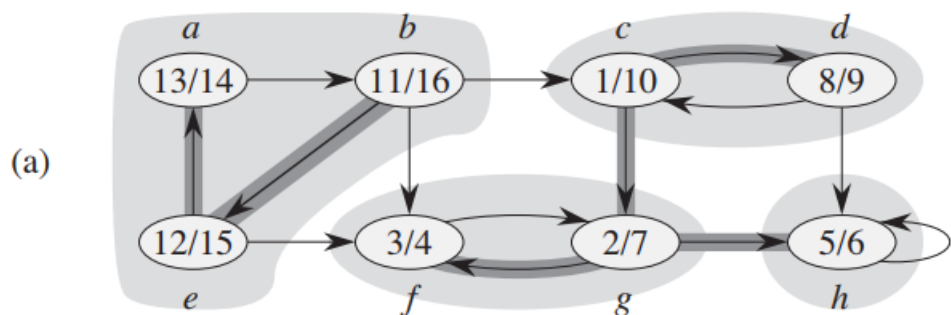
```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
  
```

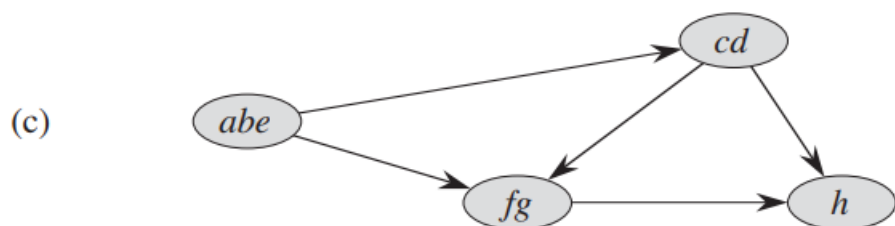
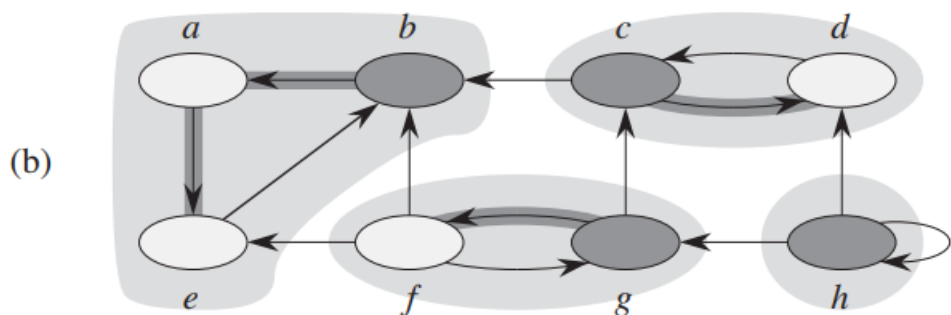



اجزای قویا همبند

اجزای قویا همبند Strongly connected components



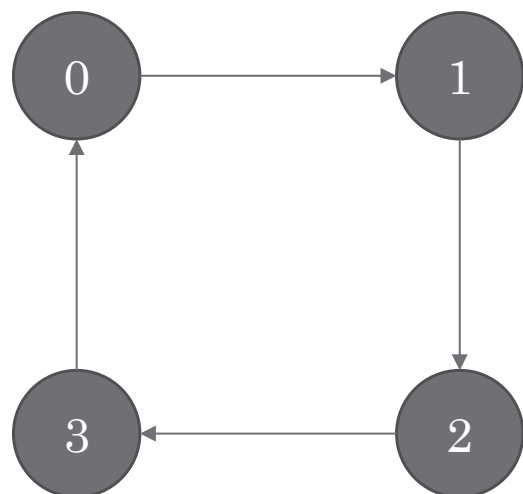
a strongly connected component of a directed graph $G = (V, E)$ is a maximal set of vertices $C \subseteq V$ such that for every pair of vertices u and v in C , we have both $u \rightsquigarrow v$ and $v \rightsquigarrow u$; that is, vertices u and v are reachable from each other.



$$G = (V, E)$$

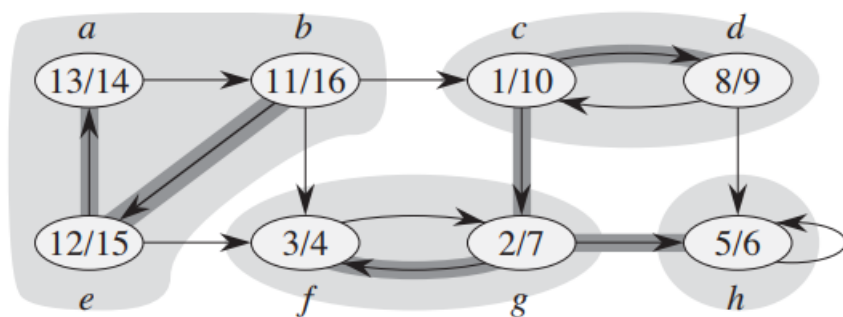
$$C \subseteq V \quad u \rightsquigarrow v \quad v \rightsquigarrow u$$

اجزای قویا همبند Strongly connected components



گراف قویا همبند

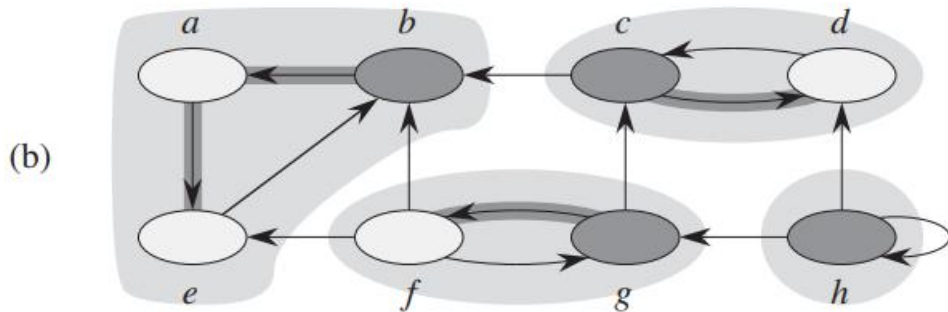
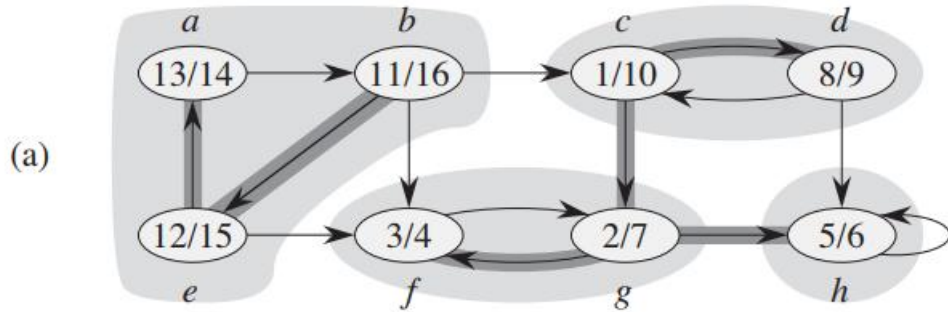
اجزای قویا همبند



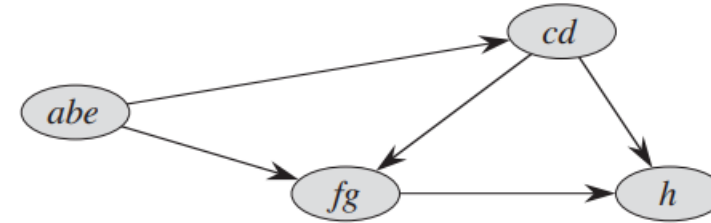
$$G = (V, E)$$

$$C \subseteq V \quad u \rightsquigarrow v \quad v \rightsquigarrow u$$

اجزای قویا همبند Strongly connected components



(c)



Algorithm uses $G^T = \text{transpose}$ of G .

- $G^T = (V, E^T)$, $E^T = \{(u, v) : (v, u) \in E\}$.
- G^T is G with all edges reversed.

Can create G^T in $\Theta(V + E)$ time if using adjacency lists.

STRONGLY-CONNECTED-COMPONENTS (G)

- 1 call $\text{DFS}(G)$ to compute finishing times $u.f$ for each vertex u
- 2 compute G^T
- 3 call $\text{DFS}(G^T)$, but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

$\text{DFS}(G)$

```

1  for each vertex  $u \in G.V$ 
2     $u.color = \text{WHITE}$ 
3     $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == \text{WHITE}$ 
7       $\text{DFS-VISIT}(G, u)$ 

```

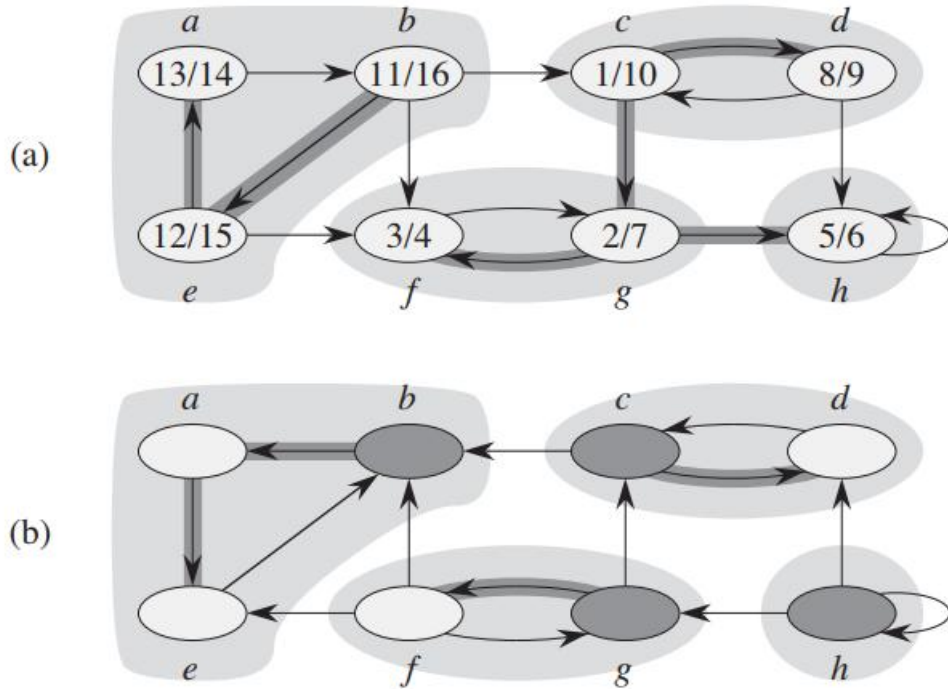
$\text{DFS-VISIT}(G, u)$

```

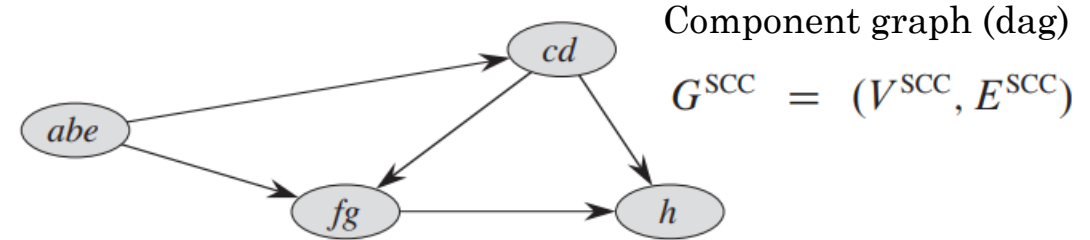
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == \text{WHITE}$ 
6       $v.\pi = u$ 
7       $\text{DFS-VISIT}(G, v)$ 
8   $u.color = \text{BLACK}$ 
9   $time = time + 1$ 
10  $u.f = time$ 

```

اجزای قویا همبند Strongly connected components



(c)



STRONGLY-CONNECTED-COMPONENTS (G)

- 1 call $DFS(G)$ to compute finishing times $u.f$ for each vertex u
- 2 compute G^T
- 3 call $DFS(G^T)$, but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

$O(V + E)$

$O(V + E)$

$O(V + E)$

DFS(G)

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )
    
```

DFS-VISIT(G, u)

```

1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```


اجزای قویا همبند Strongly connected components

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$ for each vertex u
- 2 compute G^T
- 3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

Component graph (dag)

$$G^{\text{SCC}} = (V^{\text{SCC}}, E^{\text{SCC}})$$

Observation

G and G^T have the *same* SCC's. (u and v are reachable from each other in G if and only if reachable from each other in G^T .)

Lemma

G^{SCC} is a dag. More formally, let C and C' be distinct SCC's in G , let $u, v \in C$, $u', v' \in C'$, and suppose there is a path $u \rightsquigarrow u'$ in G . Then there cannot also be a path $v' \rightsquigarrow v$ in G .

Proof Suppose there is a path $v' \rightsquigarrow v$ in G . Then there are paths $u \rightsquigarrow u' \rightsquigarrow v'$ and $v' \rightsquigarrow v \rightsquigarrow u$ in G . Therefore, u and v' are reachable from each other, so they are not in separate SCC's. ■ (lemma)

چرایی الگوریتم؟

STRONGLY-CONNECTED-COMPONENTS (G)

- 1 call $\text{DFS}(G)$ to compute finishing times $u.f$ for each vertex u
- 2 compute G^T
- 3 call $\text{DFS}(G^T)$, but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

Idea

By considering vertices in second DFS in decreasing order of finishing times from first DFS, we are visiting vertices of the component graph in topological sort order.

To prove that it works, first deal with 2 notational issues:

- Will be discussing $u.d$ and $u.f$. These always refer to *first* DFS.
- Extend notation for d and f to sets of vertices $U \subseteq V$:
 - $d(U) = \min_{u \in U} \{u.d\}$ (earliest discovery time)
 - $f(U) = \max_{u \in U} \{u.f\}$ (latest finishing time)

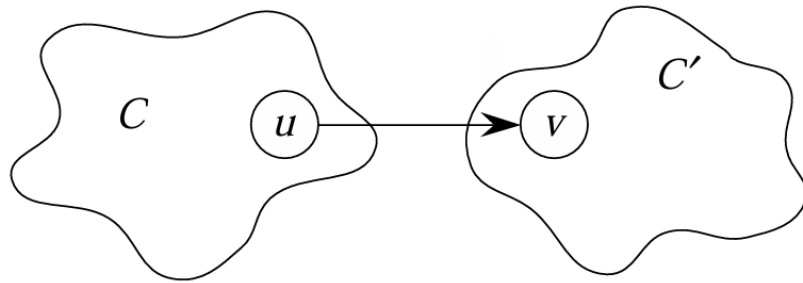
چرایی الگوریتم؟

STRONGLY-CONNECTED-COMPONENTS (G)

- 1 call $\text{DFS}(G)$ to compute finishing times $u.f$ for each vertex u
- 2 compute G^T
- 3 call $\text{DFS}(G^T)$, but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

Lemma

Let C and C' be distinct SCC's in $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ such that $u \in C$ and $v \in C'$.

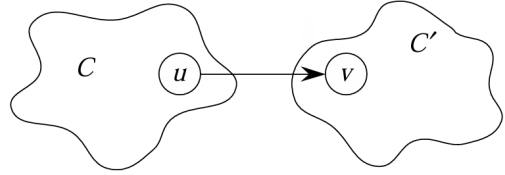


Then $f(C) > f(C')$.

چرایی الگوریتم؟

Lemma

Let C and C' be distinct SCC's in $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ such that $u \in C$ and $v \in C'$.



Then $f(C) > f(C')$.

Proof Two cases, depending on which SCC had the first discovered vertex during the first DFS.

- If $d(C) < d(C')$, let x be the first vertex discovered in C . At time $x.d$, all vertices in C and C' are white. Thus, there exist paths of white vertices from x to all vertices in C and C' .

By the white-path theorem, all vertices in C and C' are descendants of x in depth-first tree.

By the parenthesis theorem, $x.f = f(C) > f(C')$.

- If $d(C) > d(C')$, let y be the first vertex discovered in C' . At time $y.d$, all vertices in C' are white and there is a white path from y to each vertex in $C' \Rightarrow$ all vertices in C' become descendants of y . Again, $y.f = f(C')$.

At time $y.d$, all vertices in C are white.

By earlier lemma, since there is an edge (u, v) , we cannot have a path from C' to C .

So no vertex in C is reachable from y .

Therefore, at time $y.f$, all vertices in C are still white.

Therefore, for all $w \in C$, $w.f > y.f$, which implies that $f(C) > f(C')$.

■ (lemma)

چرایی الگوریتم؟

Corollary

Let C and C' be distinct SCC's in $G = (V, E)$. Suppose there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proof $(u, v) \in E^T \Rightarrow (v, u) \in E$. Since SCC's of G and G^T are the same, $f(C') > f(C)$. ■ (corollary)

Corollary

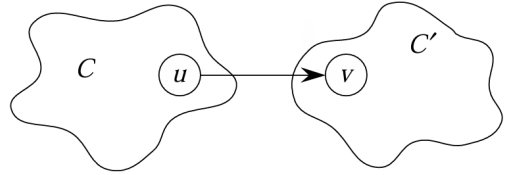
Let C and C' be distinct SCC's in $G = (V, E)$, and suppose that $f(C) > f(C')$. Then there cannot be an edge from C to C' in G^T .

Proof It's the contrapositive of the previous corollary. ■

چرایی الگوریتم؟

Lemma

Let C and C' be distinct SCC's in $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ such that $u \in C$ and $v \in C'$.



Then $f(C) > f(C')$.

Now we have the intuition to understand why the SCC procedure works.

When we do the second DFS, on G^T , start with SCC C such that $f(C)$ is maximum. The second DFS starts from some $x \in C$, and it visits all vertices in C . Corollary says that since $f(C) > f(C')$ for all $C' \neq C$, there are no edges from C to C' in G^T .

Therefore, DFS will visit *only* vertices in C .

Which means that the depth-first tree rooted at x contains *exactly* the vertices of C .

The next root chosen in the second DFS is in SCC C' such that $f(C')$ is maximum over all SCC's other than C . DFS visits all vertices in C' , but the only edges out of C' go to C , *which we've already visited*.

Therefore, the only tree edges will be to vertices in C' .

We can continue the process.

Each time we choose a root for the second DFS, it can reach only

- vertices in its SCC—get tree edges to these,
- vertices in SCC's *already visited* in second DFS—get *no* tree edges to these.

We are visiting vertices of $(G^T)^{\text{SCC}}$ in reverse of topologically sorted order.